

STEP 1: Import Required Libraries

```
# Import NumPy for numerical and vector operations
import numpy as np

# Install gensim if not already installed
!pip install gensim

# Import gensim for loading and working with pre-trained word embeddings
import gensim

# Import gensim downloader to easily load Word2Vec and GloVe models
import gensim.downloader as api

# Import matplotlib for plotting and visualization
import matplotlib.pyplot as plt

# Import PCA for dimensionality reduction (high-dim → 2D)
from sklearn.decomposition import PCA
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.0)
```

STEP 2: Load Pre-trained Embeddings

```
!wget https://github.com/mmihaltz/word2vec-GoogleNews-vectors/raw/master/GoogleNews-vectors-negative300.bin.gz

--2026-02-10 04:38:30-- https://github.com/mmihaltz/word2vec-GoogleNews-vectors/raw/master/GoogleNews-vectors-negative300.b
Resolving github.com (github.com)... 140.82.113.4
Connecting to github.com (github.com)|140.82.113.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://media.githubusercontent.com/media/mmihaltz/word2vec-GoogleNews-vectors/master/GoogleNews-vectors-negative300.bin.gz
--2026-02-10 04:38:30-- https://media.githubusercontent.com/media/mmihaltz/word2vec-GoogleNews-vectors/master/GoogleNews-vectors-negative300.bin.gz
Resolving media.githubusercontent.com (media.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ..
Connecting to media.githubusercontent.com (media.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1647046227 (1.5G) [application/octet-stream]
Saving to: 'GoogleNews-vectors-negative300.bin.gz.4'

GoogleNews-vectors- 100%[=====] 1.53G 17.8MB/s in 46s

2026-02-10 04:39:17 (34.0 MB/s) - 'GoogleNews-vectors-negative300.bin.gz.4' saved [1647046227/1647046227]
```

STEP 3: Word Similarity Computation

```
# Create a list of word pairs for similarity comparison
word_pairs = [
    ("doctor", "nurse"),
    ("cat", "dog"),
    ("car", "bus"),
    ("king", "queen"),
    ("apple", "banana"),
    ("teacher", "student"),
    ("man", "woman"),
    ("city", "village"),
    ("computer", "keyboard"),
    ("sun", "moon")
]

# Loop through each word pair
for w1, w2 in word_pairs:

    # Compute cosine similarity between the two word vectors
    similarity = word2vec_model.similarity(w1, w2)

    # Print the similarity score formatted to 3 decimal places
    print(f"{w1} - {w2}: {similarity:.3f}")
```

```
doctor - nurse: 0.632
cat - dog: 0.761
car - bus: 0.469
king - queen: 0.651
apple - banana: 0.532
```

```
teacher - student: 0.630
man - woman: 0.766
city - village: 0.479
computer - keyboard: 0.396
sun - moon: 0.426
```

STEP 4: Nearest Neighbor Exploration

```
# Load the pre-trained Word2Vec model using gensim.downloader to ensure it's available for this cell.
# This method handles downloading and loading the model correctly.
import gensim.downloader as api
word2vec_model = api.load('word2vec-google-news-300')

# Define a list of words whose nearest neighbors we want to explore
words = ["king", "university", "doctor", "computer", "india"]

# Loop through each selected word
for word in words:

    # Print the current word being analyzed
    print(f"\nTop similar words for '{word}':")

    # Retrieve the top 5 most similar words using cosine similarity
    similar_words = word2vec_model.most_similar(word, topn=5)

    # Loop through each similar word and its similarity score
    for neighbor, score in similar_words:

        # Print the neighbor word and its similarity score
        print(f" {neighbor} ({score:.3f})")
```

Top similar words for 'king':

```
kings (0.714)
queen (0.651)
monarch (0.641)
crown_prince (0.620)
prince (0.616)
```

Top similar words for 'university':

```
universities (0.700)
faculty (0.678)
university (0.676)
undergraduate (0.659)
univeristy (0.659)
```

Top similar words for 'doctor':

```
physician (0.781)
doctors (0.748)
gynecologist (0.695)
surgeon (0.679)
dentist (0.679)
```

Top similar words for 'computer':

```
computers (0.798)
laptop (0.664)
laptop_computer (0.655)
Computer (0.647)
com_puter (0.608)
```

Top similar words for 'india':

```
indian (0.697)
usa (0.684)
pakistan (0.682)
chennai (0.668)
america (0.659)
```

STEP 5: Word Analogy Tasks

```
# Define analogy tasks using positive and negative word sets
analogies = [
    (["king", "woman"], ["man"]),
    (["paris", "india"], ["france"]),
    (["teacher", "hospital"], ["school"])
]

# Loop through each analogy task
for positive_words, negative_words in analogies:

    # Solve the analogy using vector arithmetic
    result = word2vec_model.most_similar(
        positive=positive_words,    # Words to be added
        negative=negative_words,    # Words to be subtracted
```

```

topn=1                                # Return only the best match
)

# Print the analogy result
print(f"{positive_words} - {negative_words} = {result}")

['king', 'woman'] - ['man'] = [('queen', 0.7118193507194519)]
['paris', 'india'] - ['france'] = [('chennai', 0.5442505478858948)]
['teacher', 'hospital'] - ['school'] = [('Hospital', 0.6331106424331665)]

```

STEP 6: Visualization Using PCA

```

# Select words to visualize in 2D space
selected_words = [
    "king", "queen", "man", "woman",
    "paris", "france", "india", "delhi",
    "doctor", "nurse", "cat", "dog"
]

# Extract word vectors for the selected words
vectors = np.array([word2vec_model[word] for word in selected_words])

# Create a PCA object to reduce vectors to 2 dimensions
pca = PCA(n_components=2)

# Apply PCA transformation to the word vectors
reduced_vectors = pca.fit_transform(vectors)

# Create a figure for plotting
plt.figure(figsize=(8, 6))

# Loop through each word and its reduced vector
for i, word in enumerate(selected_words):

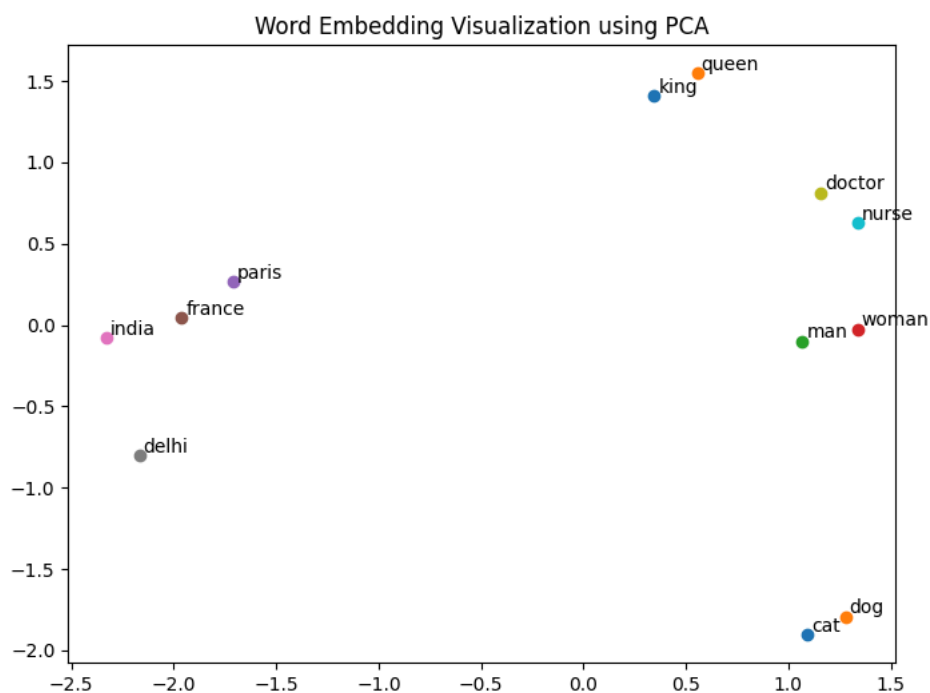
    # Plot the word as a point in 2D space
    plt.scatter(reduced_vectors[i, 0], reduced_vectors[i, 1])

    # Annotate the point with the word label
    plt.text(
        reduced_vectors[i, 0] + 0.02,
        reduced_vectors[i, 1] + 0.02,
        word
    )

# Add a title to the plot
plt.title("Word Embedding Visualization using PCA")

# Display the plot
plt.show()

```



STEP 7: Reflection and Interpretation

Word embeddings learn semantic meaning by analyzing patterns of word co-occurrence in large text corpora. Words that appear in similar contexts are represented by vectors that are close to each other in the embedding space. Similarities such as cat–dog, doctor–nurse, and king–queen make intuitive sense because these words share related contexts. Embeddings also perform well on analogy tasks by capturing relationships like gender and geography through vector arithmetic. However, embeddings sometimes behave strangely when dealing with rare words or words with multiple meanings. Since each word has only one vector, embeddings cannot fully capture different contexts of the same word, such as “bank” meaning a riverbank or a financial institution. Biases present in training data may also be reflected in similarity results. Context strongly influences vector meaning because embeddings are learned from surrounding words. Overall, word embeddings provide powerful but imperfect representations of language.

STEP 8: Lab Report

This lab focused on understanding and exploring pre-trained word embeddings using Word2Vec and GloVe models. The objective was to load the models, analyze word similarities, examine nearest neighbors, and perform analogy tasks to understand how semantic meaning is captured. Word2Vec trained on the Google News dataset with 300-dimensional vectors and GloVe trained on Wikipedia and Gigaword with 100-dimensional vectors were used. Cosine similarity results showed high scores for semantically related word pairs such as cat–dog and king–queen, confirming meaningful representations. Nearest neighbor exploration revealed clear thematic clusters around words like “king” and “university.” Analogy tasks such as king – man + woman = queen were successfully solved using vector arithmetic. The embeddings learned word meaning from context, but sometimes showed limitations with ambiguous or rare words. Overall, the lab demonstrated that word embeddings are effective but imperfect representations of language used in many NLP applications.